

Group Project Briefing Spring 2026

Introduction

On-demand food delivery platforms (e.g., Uber Eats, Glovo, Deliveroo) operate as real-time marketplaces: customers place orders, restaurants accept and prepare meals, couriers pick up and deliver, and the platform continuously optimizes dispatch, ETAs, cancellations, promotions, and fraud prevention.

These systems produce high-volume streaming data: order lifecycle events, courier availability and location/status signals, restaurant prep milestones, and customer feedback. Turning this “data in motion” into business value requires stream ingestion, stream processing, windowed analytics, and low-latency visualization.

Project Objective

Design and implement a **real-time analytics pipeline** that ingests synthetic food-delivery events, processes them with stream processing (Spark Structured Streaming), stores curated data at rest, and serves live insights via a dashboard.

This directly supports course goals around streaming models, message brokers, data formats (JSON/AVRO), and analytical pipelines using windowing, watermarks, and streaming SQL concepts.

Team Setup

- You will have to organise across 10 teams of 4 to 6 students.
- Email me if you cannot find a team to join, or you will be assigned automatically after the enrolment period.
- Your team is in charge of designing a streaming solution under technical and business constraints.

Milestones

Milestone 1: Streaming Data Feed Design & Generation

1.1 Create Two Feeds

Design and implement two distinct streaming data feeds that together represent the core operational dynamics of a real-time food delivery platform.

Your design choices must be justified in the Milestone 1 documentation, including:

- why these two feeds are essential,
- what analytics they are intended to enable,
- and how their schemas support event-time processing and late data handling.

1.2 Schema & Formats

For each selected feed:

- Define an **AVRO schema** (with sensible types, enums, optional fields, and versioning considerations).
- Generate events in **JSON and AVRO**.
- Include event-time fields needed for analytics (e.g., time stamp) and identifiers for joins (e.g., order_id, restaurant_id, courier_id, zone_id). This supports later **stream-table joins / reference data** patterns.

1.3 Realism Requirements

Your generator must support:

- **Realistic distributions** (lunch/dinner peaks; weekday/weekend differences; zone-level skew).
- **Configurability** (number of restaurants/couriers, demand surges, cancellation probability, promo periods...).
- **Edge cases for streaming correctness**, including:
 - out-of-order events (late arrivals),
 - duplicates,
 - missing steps (e.g., delivered without “picked up”),
 - “impossible” durations (for anomaly detection),
 - courier going offline mid-delivery.
- These are critical to demonstrate watermarks and event-time processing later.

1.4 Deliverables

A professional Private GitHub repository, including:

- Repository README: project overview, team structure, design note describing fields, events, assumptions, and planned analytics.
- Feed generator: Python code + README
- AVRO schemas: json schema files
- Sample Data: small sample JSON + AVRO batches

Milestone 2: Stream Analytics Implementation (Processing, Storage, Insights, Dashboard)

2.1 Ingestion

Stream your two feeds into **Azure Event Hubs**.

Teams must document topic design:

- topics, partitions, keys,
- message sizing and expected throughput,
- consumer group strategy.

2.2 Stream Processing (Spark Structured Streaming)

Implement stream processing in **Spark Structured Streaming**, including:

- Parsing/validation + enrichment
- Event-time processing
- Watermarks and windowing (tumbling/hopping/sliding/session as relevant)

Output:

- **data at rest** in Parquet, stored in Azure Blob Storage
- **aggregated metrics** for dashboard consumption

Derived metrics must support Basic/Intermediate/Advance Use Cases, for example:

Use Case 1: Basic (windowed KPIs)

Implement tumbling or hopping windows

Use Case 2: Intermediate (stateful or session-based)

Implement at least one of, for example (you can come up with your own):

- Session windows for courier “active sessions” (time online until offline)
- Demand–supply health metric per zone (orders awaiting courier vs available couriers)
- Restaurant SLA monitoring: prep-time percentiles by restaurant/zone

Use Case 3: Advanced (anomaly/fraud/forecasting)

Implement at least one, for example (you can come up with your own):

- Anomaly detection on delivery time spikes (zone-level) with late data handling
- Fraud heuristics: high refunds + repeated cancellations from same user/device/zone cluster
- Near-real-time surge indicator: predict “overload zones” using recent window trends

2.4 Visualization (Dashboard)

Create a **live dashboard** using

- Streamlit (recommended) or
- PowerBI or
- Grafana/ELK.

Dashboard content suggestions:

- Live KPIs + time-series charts
- Filtering by zone and restaurant
- Health view (alerts / SLA breaches / anomalies)
- ...

2.5 Reflection & Discussion Summary

Provide a concise team reflection:

- what you learned about streaming tradeoffs
- how you designed your data to enable analytics
- what you would need to improve for production readiness (reliability, scalability, maintainability).